SatQuery AI

The 9-day plan, explained in plain language — Sep 2 to Sep 10

Nobody is 'handing off a baton' to the next person and waiting around. **Everyone works every single day, on their own piece.** The only thing that moves between people is a specific file or result, a handful of times across the whole project — not constantly, and never all six people at once. Think of it like six people cooking different parts of one meal in the same kitchen: you're always cooking something, you just occasionally pass a pan to the person next to you.

Tanaya
Data Sourcing
→
Sai
Grounding (the AI that reads the photo)
→
Sid
Query Understanding + Language
→
Lakshya
Backend (the server in the middle)
→
Krish
Frontend (the website)
→
Arka
Visualization + Demo

Why this order?

Tanaya finds photos → Sai's AI reads them into facts → Sid turns the question + facts into a written answer → Lakshya's server wires all of that together → Krish's website shows it to the user → Arka draws the highlights and rehearses the demo. That's the natural order the data actually flows in — it's not an arbitrary assignment.

How to read the timetable

Find your name's row. Each column is one day. Every box has: a short **title**, one **plain-English sentence** anyone can understand, and (in italics) the technical detail if you want it.

→ sent — something you hand off that day
← received — something you need before you can start

Most days have neither — that just means you're heads-down on your own piece.

The whole project, in six pictures

This is the same pipeline from the cover page, drawn out. Each picture on the next few pages zooms into one of these six steps.

satquery.ai — system overview

1

Get a photo

A real satellite image comes in — from Bhuvan, RSVQA, or our own test set.

● Tanaya

2

Split & classify

The AI slices the photo into a grid and labels each square: agriculture, water, etc.

● Sai

3

Turn into facts

Those labels get written into one clean record — the JSON everyone else builds on.

● Sai

4

Understand the question

The router reads the user's question and decides which kind of answer it needs.

● Sid

5

Write the answer

Gemini turns the facts + question into a plain-English answer — evidence only.

● Sid

6

Show it on the website

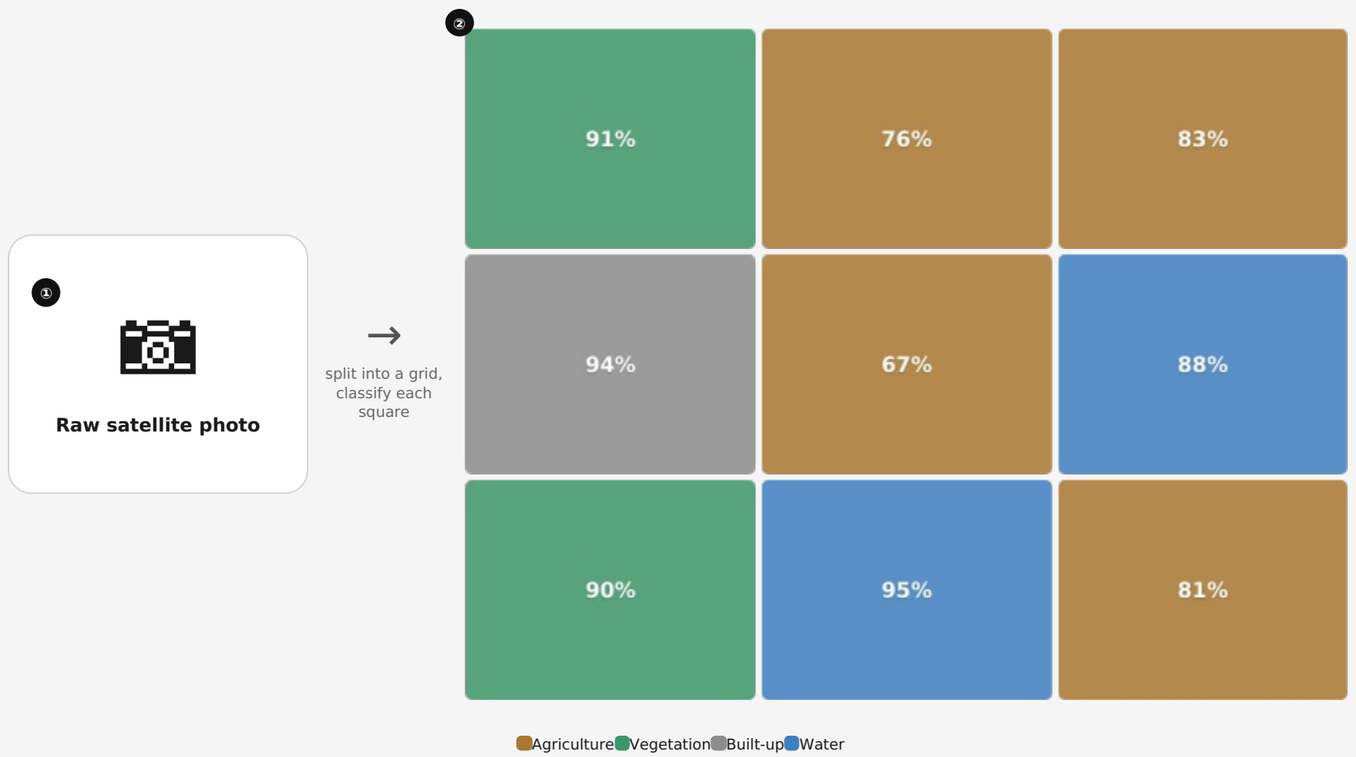
The answer, image, and highlight boxes all render on screen for the user.

● Krish + Arka

Steps 1–2: from a raw photo to a classified grid

Sai's AI doesn't look at the whole photo at once — it slices it into a grid and judges each square on its own, with a confidence score.

grounding_pipeline.py



① Raw photo

Whatever satellite image the user uploaded, untouched.

the file received by POST /query

② Classified grid

The same photo, sliced into a grid, with each square judged on its own.

grid.rows × grid.cols from grounding_contract.md

Confidence %

How sure the model is about that one square — low numbers get flagged as unreliable instead of guessed at.

each tile carries its own confidence score, 0–1

Step 3: turning the grid into structured facts

The classified grid gets written down as one tidy record — this record is the single thing every other role builds against. Think of it like a nutrition label for the photo.

grounding_contract.md — the JSON everyone builds on

① **GROUNDING FACTS** (what Sai's step hands to everyone else)

image_id sample_01

grid 3 rows × 3 columns — 9 regions total

② **tiles**

- tile_1 → **vegetation**, 91% sure
- tile_5 → **agriculture**, 67% sure
- tile_6 → **water**, 88% sure
- tile_8 → **water**, 95% sure

...one line like this for every region in the grid

③ **summary**

① image_id + grid
Which photo this is, and how many regions it was cut into.
identifies the record, matches grid.rows × grid.cols

② tiles
One line per region: what's there, and how confident the model is. This is the detailed evidence.
array of {tile_id, class, confidence, bbox}

③ summary
The tile-by-tile detail rolled up into simple percentages — this is what most answers actually quote.
percent per class, used directly in answers

Step 4: understanding what the user actually asked

Before answering anything, Sid's router reads the question and decides which of four things is really being asked — so the right logic runs next.

query_engine/router.py

① The question
Whatever the user typed into the box, exactly as they typed it.
raw string from QuestionInput.jsx

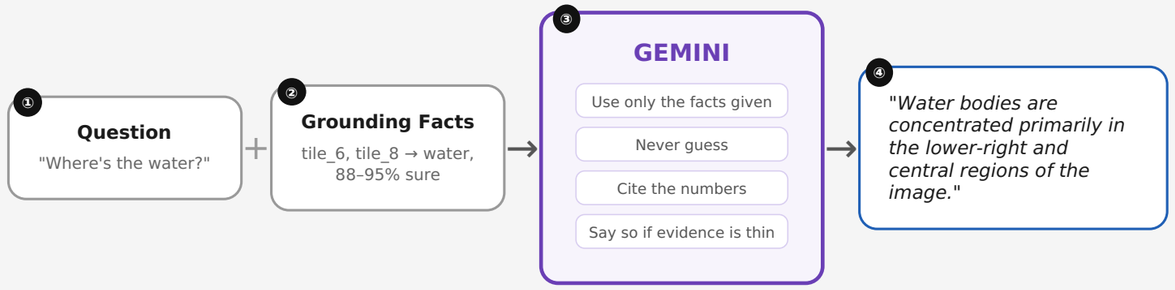
② The router
A quick classification step — no guessing about the photo, just about what TYPE of question this is.
router.py, returns an intent enum

③ Four destinations
Each path knows exactly what evidence it needs and how to phrase the answer.
intents.py: land_cover / location / change_detection / unsupported

Step 5: turning facts + question into a real answer

Gemini never sees the photo. It only ever sees the question and the structured facts card — and it's given strict rules so it doesn't make anything up.

language_service.py → Gemini API



① The question
What the user actually asked — passed through untouched.
response includes original question for context

② The facts
Only the relevant slice of evidence — not the whole photo, not the whole JSON.
Sid's prompt-builder selects relevant tiles/summary fields

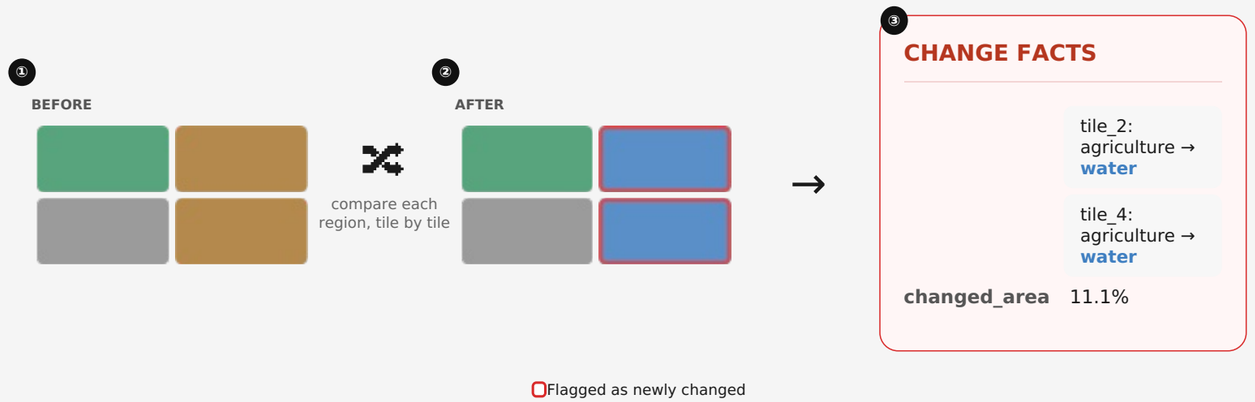
③ Gemini + rules
The model is explicitly told not to invent anything it wasn't given.
prompts.py — system prompt enforces evidence-only answers

④ The answer
A plain sentence a person can read — this is what shows up in the Answer panel.
response.answer, rendered by AnswerPanel.jsx

Bonus step: how the before/after comparison works

For the flood demo, Sai's model runs twice — once per photo — then compares the two grids region by region and flags anything that flipped to water.

change_detection/diff.py



① Before grid

The same photo, classified before the event we're checking for.

grounding_pipeline.py run #1 → before.json

② After grid

The second photo, same location, same grid — classified independently.

grounding_pipeline.py run #2 → after.json

③ Change facts

Only tiles that flipped to water get flagged — everything else is ignored as unchanged.

diff.py compares before.json vs after.json tile by tile

Days 1–5: building the core pipeline

Single-image question & answer, start to finish.

	Day 1 Sep 2	Day 2 Sep 3	Day 3 Sep 4	Day 4 Sep 5	Day 5 Sep 6
Tanaya Data Sourcing	**Find our practice photos** Go find real satellite images we can test on. A handful is enough — we are not downloading a whole research dataset. *Technically: pull sample tiles from Bhuvan, RSVQA and BigEarthNet into data/*	**Send Sai the first real batch** Sort the images into clearly named folders and hand them to Sai so the AI has something real to chew on. *Technically: data/demo/single/, image_0N.jpg naming* → to Sai: first labeled image batch	**Add trickier photos** Throw in some harder examples on purpose — mixed forest, water, and city — so we find out where the AI gets confused, early. → to Sai: varied-terrain set	**Start hunting for a flood example** Find a real 'before the flood' and 'after the flood' pair of photos of the same place — this powers our second demo.	**Lock in the flood pair** Make sure the before/after photos are the exact same crop and size, so a fair comparison is possible. *Technically: resize/align, verify identical dimensions* → to Sai: aligned before/after pair
Sai Grounding (the AI that reads the photo)	**Understand the existing AI model** Before writing anything, sit with the existing Colab notebook and understand exactly how it looks at a photo and labels regions — grid size, categories, everything. *Technically: document class list + grid size into grounding_contract.md*	**Get one real result out of it** Take Tanaya's first photos and get the model actually producing an output on a real image — doesn't need to be perfect yet, just real. *Technically: grounding_pipeline.py, first real grounding JSON* ← from Tanaya: first image batch → to Sid: real output, once stable → to Lakshya: real output, once stable	**Make it more accurate** Run it on the harder images Tanaya sent, see where it gets things wrong, and fix those specific mistakes. Also start sketching how a 'before vs after' comparison would work. ← from Tanaya: varied-terrain set	**Make sure it holds up under real use** Run the model through the actual website's upload path (not just a script) and make sure nothing breaks when it's called for real.	**Build the flood-comparison logic** Write the piece that takes the 'before' and 'after' results and figures out exactly which spots turned into water. *Technically: diff.py — flags tiles that flip to water_body* ← from Tanaya: aligned before/after pair → to Lakshya: change-detection results
Sid Query Understanding + Language	**Plan how questions get understood** On paper: list out the different kinds of questions someone could ask ('what's mostly here', 'where's the water', 'what changed') and how the AI should respond to each. *Technically: intents.py list + first Gemini prompt draft*	**Build the question-classifier using pretend data** Since Sai's real results aren't ready, build and test the piece that figures out which type of question was asked using made-up sample data.	**Get the real AI (Gemini) talking** Take one of Sai's real results plus a real question, send it to Gemini, and check the answer actually makes sense and isn't made up. ← from Sai: first real grounding output	**Plug the classifier into the real backend** Hand the finished question-classifier logic to Lakshya so it's part of the live chain, not just a standalone test. → to Lakshya: finished question-router	**Teach it to spot 'what changed' questions** Add the logic that recognises when someone is asking about change/flooding, so it routes to Sai's before/after comparison instead.
Lakshya Backend (the server in the middle)	**Set up the app's skeleton, agree the shared format** Get the backend server running (empty for now). Together with Krish, agree exactly what the website will send the server and what it'll get back — this is the contract everyone else builds against. *Technically: FastAPI scaffold + api_contract.md* → to Krish: agreed request/response format	**Return a pretend answer** So Krish isn't stuck waiting, make the server return a fake but correctly-shaped answer for now. → to Krish: mock response to build against	**Wire in the real AI model** Swap the pretend answer for an actual call into Sai's real model. ← from Sai: real grounding module	**Connect the whole chain — CRITICAL DAY** Photo in → Sai reads it → Sid figures out the question → Gemini writes the answer → sent back out. This is the day it all connects for the first time. ← from Sai: real grounding output ← from Sid: finished question-router → to Krish: live, working answer	**Add the before/after pathway** Build a second path in the server specifically for two-photo, 'what changed' questions. ← from Sai: change-detection results → to Krish: live change-detection answer
Krish Frontend (the website)	**Set up the website's skeleton, agree the shared format** Get a blank website running. Together with Lakshya, agree what the site will send the server and what it'll get back. *Technically: React/Vite scaffold* ← from Lakshya: agreed request/response format	**Build the question box** Build the actual text box where someone types their question, plus a loading spinner, wired up to Lakshya's pretend answer. ← from Lakshya: mock response	**Build the photo-upload box** Build the part where someone drags in their satellite photo and sees it appear on screen.	**Connect the website to the real, live server** Swap out every pretend response for the real, fully-working chain from Lakshya. ← from Lakshya: live working answer	**Build the before/after screen** Build the second mode: two photo-upload boxes side by side, for change questions. ← from Lakshya: live change-detection answer
Arka Visualization + Demo	**Plan the demo story and slides** Write out, step by step, exactly what we'll show the judges, and start on the presentation slides. *Technically: demo_script.md draft*	**Sketch the highlight overlay** Draw out — on paper or Figma — how the coloured boxes will sit on top of the photo to show what the AI found.	**Start building the real overlay** Actually implement the highlight boxes inside Krish's website, using Sai's real coordinates.	**Finish the single-photo overlay** Get the land-cover highlight fully working end to end — this is the first complete visual result.	**Add the flood highlight** Extend the overlay so flooded/changed regions get highlighted on the 'after' photo.

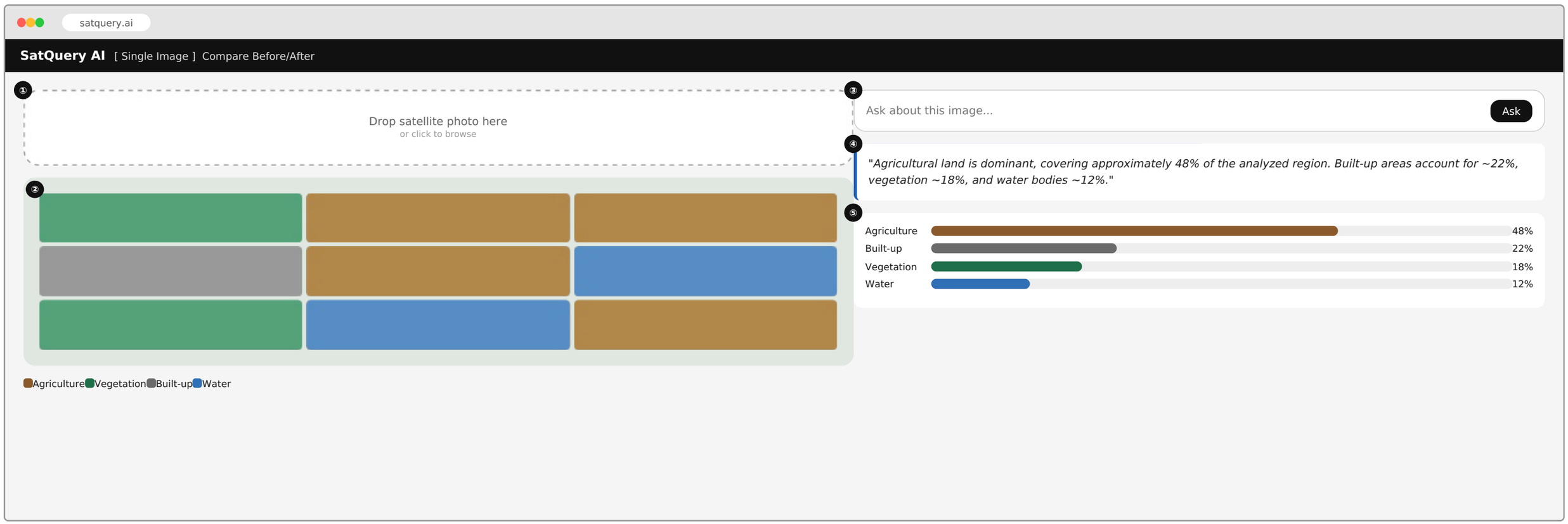
Days 6–9: flood detection, hardening, rehearsals, freeze

Adding the before/after feature, then making sure nothing breaks on stage.

	Day 6 Sep 7	Day 7 Sep 8	Day 8 Sep 9	Day 9 Sep 10
Tanaya Data Sourcing	**Deliberately supply broken photos** On purpose, hand over a blurry photo, a corrupt file, a wrong-size pair — so we can check the app fails politely instead of crashing. → to Sai: edge-case test images	**Support the rehearsal** Be on standby — if anyone needs one more image during today's live run-through, get it to them fast.	**Fix any data problems** Whatever image-related issue showed up in yesterday's rehearsal, sort it out today.	**Final sanity check** Double, triple check the exact images we will use live in front of the judges actually work.
Sai Grounding (the AI that reads the photo)	**Handle bad photos gracefully** Make sure a blurry or broken photo doesn't crash the whole app — it should just say 'not confident' instead of guessing wildly. ← from Tanaya: edge-case images	**Final accuracy pass** One more careful pass over every demo photo, so we know exactly which ones are reliable for the live demo. → to Arka: notes on which images are safe to demo	**Fix anything from last rehearsal** Patch whatever broke during yesterday's dry run. If there's time to spare, try a second flood example as a backup.	**Lock it in** Confirm the model runs cleanly, start to finish, with no last-minute changes.
Sid Query Understanding + Language	**Handle confusing questions gracefully** Make sure a totally unrelated question ('what's my favourite colour?') gets a polite 'I can only answer about this image' instead of a made-up answer.	**Make the answers sound better** Polish the wording Gemini uses so answers are clear and explain themselves — judges will care about this.	**Fix anything from last rehearsal** Patch whatever broke in yesterday's dry run.	**Lock it in** Confirm exactly how the AI phrases its final answers, and don't touch it again.
Lakshya Backend (the server in the middle)	**Make failures polite, not crashes** If a photo is bad, the AI times out, or a question is nonsense — make sure the server sends back a clear, readable error instead of breaking.	**Speed things up** Make responses faster so the live demo doesn't feel laggy in front of judges.	**Fix anything from last rehearsal** Patch whatever broke in yesterday's dry run.	**Lock the server, final version** Merge the finished code into the main version. No more changes after this.
Krish Frontend (the website)	**Design clean error messages** Show a friendly message on screen for every kind of failure Lakshya can now report.	**Polish the look, make it responsive** Clean up spacing and sizing so it looks sharp on the actual demo screen/projector.	**Fix anything from last rehearsal** Patch whatever broke in yesterday's dry run.	**Lock the website, final version** Merge the finished code into the main version. No more changes after this.
Arka Visualization + Demo	**Stress-test 'no real change'** Feed it two nearly identical before/after photos and make sure it does NOT falsely claim flooding happened.	**Run the first full live rehearsal** Get everyone in a room, run the whole thing live start to finish, and write down every single thing that goes wrong. ← from Tanaya: rough edges ← from Sai: rough edges ← from Sid: rough edges ← from Lakshya: rough edges ← from Krish: rough edges	**Polish visuals, run rehearsal #2** Clean up the visual highlights, then time a second full run-through.	**Lead the final rehearsal** Check the slides match what the app actually does, then run the last timed practice with everyone.

What the website actually looks like — Mode 1: Single Image

This is what Krish is building and what Arka's overlay sits inside of. Numbers below match the numbers on the mockup.



① Upload area

Where the user drops in their satellite photo.

React dropzone component, ImageUploader.jsx

② Photo with highlight grid

Once analyzed, coloured boxes appear over the regions the AI classified.

OverlayCanvas — SVG rectangles positioned from each tile's bbox

③ Question box

Where the user types their question and hits Ask.

QuestionInput.jsx, calls POST /query

④ Answer panel

The plain-English answer Gemini generated from the evidence.

AnswerPanel.jsx, renders response.answer

⑤ Evidence bars

A simple bar per land-cover type, so the answer isn't just a claim — you can see the numbers behind it.

renders response.overlay_data.summary

What the website actually looks like — Mode 2: Compare Before/After

Same website, a second mode for the flood/change-detection demo.

① Before / After photos

Two photos side by side. Any region flagged as newly changed gets a highlighted box on the After photo.

BeforeAfterViewer.jsx + ChangeOverlay

② Question box

Same question box, but now it's asking about the pair of photos.

calls POST /query/change

③ Answer panel

A plain-English explanation of what changed and roughly how much.

renders response.answer + summary.changed_area_percent

In one paragraph: what YOU are actually building

Tanaya

Data Sourcing

You are the one making sure everyone else has real, good photos to work with — without you, Sai has nothing to test on and the whole thing is guesswork.

You know you're done when: the demo photos are final, verified, and everyone who needs one has it.

Sai

Grounding (the AI that reads the photo)

You are building the part that actually 'reads' a satellite photo and turns it into facts a computer (and later, a person) can use — what's in each region, how confident the model is, and later, what changed between two photos.

You know you're done when: any demo photo goes in and a clean, reliable result comes out, every time.

Sid

Query Understanding + Language

You are the translator between a human's messy question and the AI's structured facts — you decide what kind of question it is, and you're the one making sure the final answer sounds like a person wrote it, not a spreadsheet.

You know you're done when: every kind of question (including weird ones) gets a sensible, honest answer.

Lakshya

Backend (the server in the middle)

You are the middleman everyone else's work has to pass through — the server that takes a photo and question from the website, runs it through Sai and Sid's work, and sends back an answer. If your part breaks, nothing else matters.

You know you're done when: the whole chain runs end-to-end, reliably, and fails politely when something's wrong.

Krish

Frontend (the website)

You are building the actual thing the judges will click on — the website itself. Everything everyone else builds is invisible unless it shows up correctly on your screen.

You know you're done when: someone can upload a photo, ask a question, and see a clear answer — with zero confusion.

Arka

Visualization + Demo

You are making the AI's thinking visible — the coloured highlights on the photo — and you're the one who makes sure the live demo actually goes smoothly in front of judges.

You know you're done when: the highlights are accurate and the whole team can run the demo without you saying a word.